

GH - 300

Copilot Customization

Prompt Files · Skills · Agents · Hooks

Turn one Copilot into your team's Copilot — encoded, reviewed, and shared in the repo.

Prompt files

Agent Skills

Custom Agents

Hooks

Awesome Copilot

Sandeep Soni · Deccansoft Software Services Pvt Ltd

Deck 10 · GH-300 Certified GitHub Copilot Expert Training

The customisation landscape — four primitives, one repo

Custom instructions teach Copilot HOW to write (covered in Deck 3). These four primitives extend WHAT it can do, and as WHO.

Prompt files

Reusable, parameterised prompts saved as `.prompt.md`. Invoke as slash commands. Best for repeatable tasks (scaffold component, security audit, release notes).

Agent Skills

Folders of instructions + scripts + resources. Loaded on demand when the description matches the task. Best for specialised workflows.

Custom Agents

`.agent.md` personas with scoped tools, pinned models, and handoffs. Best for role-based work (Planner → Coder → Reviewer).

Hooks

Pre / post-processing logic attached to a specific agent. Best for validation, auditing, and integrating downstream systems.

Prompt Files — reusable prompts as slash commands

Markdown format

.prompt.md files in .github/prompts/. Optional YAML frontmatter + Markdown body.

Slash commands

Type / in chat → autocomplete shows your prompts → invoke instantly.

Shared in repo

Live alongside your code. Reviewed in PRs. Same prompt across the whole team.

Configurable

Pin agent mode, model, tools, description, and argument hints via frontmatter.

Availability: VS Code, Visual Studio, and JetBrains IDEs. github.com chat does not currently support slash-invoked prompt files.

Anatomy of a prompt file — frontmatter + body

```
.github/prompts/reactform.prompt.md
```

```
---
agent: 'agent'
model: 'Claude Sonnet 4.6'
tools: ['githubRepo', 'codebase']
description: 'Generate a React form'
argument-hint: 'mention form name'
---
```

Your goal: generate a new React form component based on our templates.

Requirements:

- * Use design-system components from [Form.md](../docs/design-system/Form.md)
- * Use react-hook-form for state
- * Use yup for validation
- * Always define TypeScript types

If the form name is missing, ask first.

Frontmatter fields

agent

"agent" (agentic) or "ask" (Q&A)

model

Pin the LLM (GPT-5.2-Codex, Claude Sonnet 4.6, ...)

tools

Array of tools / MCP servers to enable

description

Shown in the slash-command menu

argument-hint

Placeholder text for extra input

Variables in prompt files — make them flexible

Workspace

`${workspaceFolder}`

Path to the workspace / project root

Selection

`${selection}`

Currently selected code in the editor

`${selectedText}`

Text content of the current selection

File

`${file}`

Path to the active file

`${fileDirectory}`

Directory of the active file

Example: "Review the code in `${file}` focusing on `${input:reviewFocus}`" — flexible without rewriting the prompt.

Input

Where prompt files pay off — six high-leverage use cases

Scaffold components

Consistent React / Angular / Vue components from one template.

Run & fix tests

Execute tests + fix failures with project context.

Prepare pull requests

PR descriptions, changelogs, reviewer checklists.

Security reviews

Standardised audit checklist applied across the team.

Generate documentation

API docs, READMEs, ADRs from code + context.

Override agent behaviour

Minimal-implementation plans, API mockups, dry-run prompts.

Example — React form generator

reactform .prom pt.m d

```
---
agent: 'agent'
model: 'Claude Sonnet 4.6'
tools: ['githubRepo', 'search/codebase']
description: 'Generate a new React form'
argument-hint: 'form name + fields'
---
```

Generate a React form component using the templates in #tool:githubRepo contoso/react-templates.

Ask for the form name and fields if they are not provided.

Requirements:

- * design-system components
- * react-hook-form for state mgmt
- * yup validation; TS types always
- * reusable validation schemas

How it works

Agent mode — runs in full agentic mode; can create files, read repos, search.

Model pinned — consistent generations across the team.

Tools enabled — githubRepo + codebase search for context-aware output.

Smart fallback — asks for the form name and fields if missing.

Standards enforced — react-hook-form + yup + TS, every time.

Example — REST API security review

securityreview.prompt.md

```
---  
agent: 'ask'  
model: 'Claude Sonnet 4.6'  
description: 'REST API security review'  
---
```

Perform a REST API security review
and provide a Markdown TODO list.

Check:

- * All endpoints protected by authentication + authorisation.
- * All inputs validated and sanitised.
- * Rate limiting / throttling in place.
- * Logging + monitoring for sec events.

Return the TODO grouped by priority
and issue type. Cite line numbers.

Key differences from Example 1

Ask mode (agent: ask) — analyses and reports rather than editing files.

No tools needed — uses only the codebase context the user provides.

Structured output — explicitly asks for a Markdown TODO grouped by priority + type.

Team benefit — every developer uses the same security checklist on every PR.

Testing a prompt file safely before sharing

1

Start in Ask mode

Set agent: ask while iterating. No file writes; no surprises.

2

Sandbox the inputs

Test on a throwaway branch or a sample repo first. Never iterate against main.

3

Run the play button

In VS Code, use the editor "Play" button on the prompt file. Runs in a scratch chat.

4

Inspect the references

Open the References list in the response. Confirm which files / tools were actually used.

5

Diff outputs across models

Pin different models in turn (Claude / GPT) — compare. Pin the one that wins consistently.

6

Promote with a name

Rename from draft-foo.prompt.md to foo.prompt.md only when you trust it. Commit.

Red flags before you commit

Tools that should never fire for this task are in the tools list — delete them. The prompt edits files when agent: ask is enough. The instructions reference docs / files that no longer exist. Outputs vary wildly across runs — your prompt is under-specified.

Agent Skills — folders of instructions, scripts, and resources

A specialised capability that loads on demand. Like a prompt file with attached files, scripts, and templates — and an open standard honored across AI tools.

Language / framework expertise	Architecture rules	Coding standards	Security practices
Testing strategy	Cloud / infrastructure	Performance expectations	Doc generation pipelines

Where they live

`.github/skills/<skill-name>/SKILL.md` • `.agents/skills/<skill-name>/SKILL.md` • `.claude/skills/` (Claude Code compatible)

Same folder structure across tools — write once, run anywhere AGENTS-style.

Creating an Agent Skill — directory + SKILL.md frontmatter

directory layout

```
.github/skills/  
webapp-testing/  
  SKILL.md      # instructions  
  test-template.js # template  
  examples/     # samples  
    login-test.js  
    search-test.js
```

SKILL.md

```
---  
name: webapp-testing  
description: Guide for testing  
  web apps using Playwright.  
  Use when asked to create  
  or run browser-based tests.  
---  
  
# Web Application Testing  
[Instructions, examples, guidelines]
```

Skill body should include

- What the skill helps accomplish (one paragraph).
- When to use the skill (concrete triggers — Copilot uses these to decide relevance).
- Step-by-step procedures to follow.
- Examples of expected input / output — with the input wrapped in a clear delimiter.
- References to included scripts / resources (relative paths).

How Copilot uses skills — discover, load, fetch resources

1

Discovery

Copilot reads the name + description from every skill's YAML frontmatter. Lightweight scan — cheap, runs always.

2

Loading

When your request matches a skill's description, Copilot loads the full SKILL.md body into context.

3

Resources

Additional files (scripts, templates, examples) are loaded ONLY when referenced. Keeps the token budget lean.

On-demand loading keeps your context window efficient — only relevant skills consume tokens. Description quality is everything; write it to be matchable.

Skills vs Prompts vs Instructions — when to use which

Feature	Agent Skills	Prompt Files	Custom Instructions
Purpose	Specialised capability	Reusable prompt as slash cmd	Coding standards always on
Loading	On demand (description match)	Explicit invocation	Always in context
Content	Instructions + scripts + files	Markdown + frontmatter	Markdown only
Portability	VS Code, CLI, cloud agent	VS Code, VS, JetBrains	VS Code + github.com
Standard	Open (agentskills.io)	VS Code / GitHub-specific	GitHub-specific

Use together:

Custom Instructions for always-on standards (naming, formatting, architecture). Prompt Files for repeated team tasks (scaffold, audit). Agent Skills for specialised workflows (browser testing, IaC review) that load only when relevant.

Hooks — pre / post-processing attached to a specific agent

Agent-scoped hooks (Apr 2026 preview) let you attach logic that runs before or after a custom agent's turn. Lets you validate, audit, redact, or trigger downstream systems — without changing the agent's instructions.

preInvoke

Runs before the agent receives your prompt. Validate, sanitise, or enrich the input.

postInvoke

Runs after the agent responds. Log, transform, redact, or notify downstream systems.

onToolCall

Wraps any tool invocation. Useful for an org-wide audit trail across MCP tools.

onError

Runs when the agent loop fails. Capture context and route to your error-tracking tool.

`.agent.m d • hooks`

```
# .github/agents/bug-hunter.agent.m d (frontmatter excerpt)
hooks:
  preInvoke: scripts/hooks/redact-pii.ts # strip PII from incoming traces
  postInvoke: scripts/hooks/log-to-splunk.ts # audit trail for compliance
```

Hooks — where they earn their keep

Compliance audit

postInvoke logs every agent interaction to Splunk / Datadog with user, agent name, model, tools used, and tokens consumed.

PII redaction

preInvoke strips card numbers, emails, and tokens from incoming traces before the model ever sees them.

Output validation

postInvoke runs a JSON Schema check or a lint pass on generated code; rejects and re-prompts on violations.

Notification fan-out

postInvoke posts a Slack message when the Bug-Hunter finishes; opens a Linear ticket when severity is high.

Rate limiting

preInvoke checks a team-level budget; refuses the invocation when the daily token cap is reached.

Style enforcement

postInvoke runs Prettier / dotnet-format on any files the agent edited; auto-commits the formatting pass.

awesome-copilot — community-curated starting points

What it is

- [github/awesome-copilot](#) — community-contributed instructions, prompts, skills, and agents.
- Searchable + filterable index across hundreds of resources.
- Tools section for MCP servers and dev tooling.
- Learning Hub with guides and tutorials.

How to use it well

- Start with a vetted prompt / skill, then adapt to your repo conventions.
- Treat it as a reference for HOW to write — not a paste-into-production source.
- Pin to a specific commit when you copy — community entries do drift.
- Contribute back: your domain-specific skill is someone else's starting line.

Key takeaways

1

Four primitives, one repo

Prompt files · Agent Skills · Custom Agents · Hooks. Layer with Custom Instructions (Deck 3) to teach Copilot HOW + WHAT + WHO.

2

Prompt files = reusable slash cmds

.github/prompts/*.prompt.md. Frontmatter pins mode, model, and tools. Body sets the goal.

3

Test before sharing

Ask mode → sandbox → play button → check references → diff across models → rename from draft.

4

Skills load on demand

Discovery on description, body on match, resources on reference. Token-efficient by design.

5

Custom Agents enforce least privilege

.agent.md persona with scoped tools — a planner that literally cannot edit code. Handoffs chain roles safely.

6

Hooks close enterprise loops

preInvoke / postInvoke / onToolCall / onError — compliance, redaction, validation, notification.

Thank you.

Bring your repo — we will encode its conventions together.

Sandeep Soni · Deccansoft Software Services Pvt Ltd

GH-300 · Copilot Customization · Deck 10